Veo 3.1 API 使用指南

Veo 3.1 是 Google DeepMind 发布的新一代视频生成模型，支持通过文本或图像提示生成高质量、高分辨率的视频内容。本指南介绍如何通过 **new-api 中转平台**调用 Veo 3.1 API，完成从提交任务到获取视频的完整流程。

> 本文档基于 new-api 源码（`relay/channel/task/gemini/`）整理，所有接口路径、参数名称均以实际代码实现为准。

目录

1. 准备工作
2. API 基础信息
3. 发起视频生成任务
4. 获取任务状态
5. 获取视频内容
6. 完整示例代码
7. 错误处理
8. 最佳实践

准备工作

获取 API Key

在 new-api 平台注册账号后，进入控制台 → **令牌管理** → 创建新令牌，获取你的 `API Key`。

安装依赖

```
# Node.js
npm install axios

# Python
pip install httpx
```

API 基础信息

项目	值
中转 Base URL	`https://your-new-api-domain.com/v1`
模型名称	`veo-3.1-generate-preview` 或 `veo-3.1-fast-generate-preview`
认证方式	`Authorization: Bearer <API_KEY>`
内容类型	`application/json`

> 将 `your-new-api-domain.com` 替换为你实际使用的 new-api 中转平台域名。

发起视频生成任务

Veo 3.1 采用**异步任务**模式：提交请求后返回任务 ID，需要轮询该 ID 获取生成结果。

接口

```
POST /v1/video/generations
```

> **注意**：路径是 `/v1/video/generations`（`video` 为单数），不是 `/v1/videos/generations`。

请求参数（TaskSubmitReq）

参数	类型	必填	说明
`model`	string	是	模型名称，如 `veo-3.1-generate-preview`
`prompt`	string	是	视频描述文本，建议英文，详细描述场景、风格、镜头运动
`image`	string	否	单张参考图像（`...` 格式），用于图生视频
`images`	string[]	否	多张参考图像数组（base64 或 data URI）
`size`	string	否	视频尺寸，格式 `宽x高`，如 `1920x1080`、`1080x1920`。平台自动推算分辨率（`720p` / `1080p` / `4k`）和宽高比（`16:9` / `9:16`）
`duration`	int	否	视频时长（秒），默认 `8`
`metadata`	object	否	Veo 原生参数，见下表，优先级高于顶层字段

metadata 对象字段（Veo 原生参数）

字段	类型	说明
`durationSeconds`	int	视频时长（秒），默认 `8`，覆盖顶层 `duration`
`aspectRatio`	string	宽高比，如 `16:9`、`9:16`。覆盖由 `size` 推算的值
`resolution`	string	分辨率，如 `720p`、`1080p`、`4k`。覆盖由 `size` 推算的值
`negativePrompt`	string	负面提示词
`seed`	int	随机种子，相同 seed 可复现结果
`generateAudio`	bool	是否生成音频（Veo 3.1 支持）
`personGeneration`	string	人物生成模式
`compressionQuality`	string	压缩质量

> **图像输入说明：** `image` / `images` 字段目前仅支持 **base64 编码**（原始 base64 或 `...` 格式），**不支持直接传入图片 HTTP URL**。如需图生视频，需先将图片转为 base64。

示例请求（文生视频）

```
curl -X POST https://your-new-api-domain.com/v1/video/generations \
  -H "Authorization: Bearer $API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "veo-3.1-generate-preview",
    "prompt": "A majestic eagle soaring over snow-capped mountains at golden hour, cinematic w
    "size": "1920x1080",
    "duration": 8,
    "metadata": {
      "negativePrompt": "blurry, low quality, distorted",
      "generateAudio": false
    }
  }'
```

示例请求（图生视频）

```
# 先将图片转为 base64
BASE64_IMG=$(base64 -w 0 reference.jpg)

curl -X POST https://your-new-api-domain.com/v1/video/generations \
  -H "Authorization: Bearer $API_KEY" \
  -H "Content-Type: application/json" \
  -d "{
    \"model\": \"veo-3.1-generate-preview\",
    \"prompt\": \"The scene comes alive with gentle movement, camera slowly pans right\",
    \"image\": \"${BASE64_IMG}\",
    \"size\": \"1920x1080\",
    \"duration\": 5
  }"
```

响应示例

```
{
  "id": "task-abc123xyz456",
  "task_id": "task-abc123xyz456",
  "object": "video",
  "model": "veo-3.1-generate-preview",
  "status": "queued",
  "progress": 0,
  "created_at": 1741276800
}
```

> `id` 字段即为后续轮询使用的**任务 ID**，请保存此值。

获取任务状态

视频生成通常需要 **30 秒到 3 分钟**，需要通过任务 ID 轮询状态。

接口

```
GET /v1/video/generations/{task_id}
```

示例请求

```
curl https://your-new-api-domain.com/v1/video/generations/task-abc123xyz456 \
  -H "Authorization: Bearer $API_KEY"
```

任务状态说明

状态值	含义
`queued`	任务排队中，等待处理
`in_progress`	任务处理中，继续轮询
`completed`	任务完成，可获取视频
`failed`	任务失败，查看 `error` 字段

响应示例（处理中）

```
{
  "id": "task-abc123xyz456",
  "object": "video",
  "model": "veo-3.1-generate-preview",
  "status": "in_progress",
  "progress": 50,
  "created_at": 1741276800
}
```

响应示例（已完成）

```
{
  "id": "task-abc123xyz456",
  "object": "video",
  "model": "veo-3.1-generate-preview",
  "status": "completed",
  "progress": 100,
  "created_at": 1741276800,
  "completed_at": 1741276920
}
```

获取视频内容

任务状态为 `completed` 后，通过以下接口获取视频文件（平台代理下载）。

接口

```
GET /v1/videos/{task_id}/content
```

> **注意**：此接口路径中 `videos` 为复数，与提交/查询接口（`video` 单数）不同。支持 Token 认证或 Session 认证。

示例请求

```
# 直接下载到文件
curl https://your-new-api-domain.com/v1/videos/task-abc123xyz456/content \
  -H "Authorization: Bearer $API_KEY" \
  -o output_video.mp4
```

完整示例代码

JavaScript / Node.js

```
const axios = require('axios');
const fs = require('fs');

const API_KEY = process.env.API_KEY;
const BASE_URL = 'https://your-new-api-domain.com/v1';

const client = axios.create({
  baseURL: BASE_URL,
  headers: {
    'Authorization': `Bearer ${API_KEY}`,
    'Content-Type': 'application/json',
  },
});

// 1. 发起视频生成任务
// 注意：路径是 /video/generations（video 单数）
async function createVideoTask(prompt, options = {}) {
  const body = {
    model: 'veo-3.1-generate-preview',
    prompt,
    size: options.size ?? '1920x1080',     // 格式 "宽x高"
    duration: options.duration ?? 8,        // 顶层 duration 字段（int）
  };

  // Veo 原生参数通过 metadata 传递
  const metadata = {};
  if (options.negativePrompt) metadata.negativePrompt = options.negativePrompt;
  if (options.seed ≠ null)   metadata.seed = options.seed;
  if (options.generateAudio ≠ null) metadata.generateAudio = options.generateAudio;
  if (Object.keys(metadata).length > 0) body.metadata = metadata;

  const response = await client.post('/video/generations', body);
  return response.data;
}

// 2. 轮询任务状态
// 注意：路径是 /video/generations/:task_id（video 单数）
async function pollTaskStatus(taskId, intervalMs = 5000, timeoutMs = 300000) {
  const startTime = Date.now();

  while (Date.now() - startTime < timeoutMs) {
    const response = await client.get(`/video/generations/${taskId}`);
    const task = response.data;

    console.log(`[${new Date().toISOString()}] Status: ${task.status} (${task.progress}%)`);
```

```
    if (task.status === 'completed') return task;

    if (task.status === 'failed') {
      throw new Error(`Task failed: ${task.error?.message ?? 'Unknown error'}`);
    }

    await new Promise(resolve => setTimeout(resolve, intervalMs));
  }

  throw new Error(`Task timed out after ${timeoutMs / 1000}s`);
}

// 3. 下载视频内容
// 注意：路径是 /videos/:task_id/content（videos 复数）
async function downloadVideo(taskId, outputPath) {
  const response = await client.get(`/videos/${taskId}/content`, {
    responseType: 'stream',
  });
  const writer = fs.createWriteStream(outputPath);
  response.data.pipe(writer);

  return new Promise((resolve, reject) => {
    writer.on('finish', resolve);
    writer.on('error', reject);
  });
}

// 主流程
async function generateVideo(prompt, outputPath = 'output.mp4') {
  console.log('Submitting video generation task...');
  const task = await createVideoTask(prompt, {
    size: '1920x1080',
    duration: 8,
    negativePrompt: 'blurry, low quality, distorted',
    generateAudio: false,
  });
  console.log(`Task created: ${task.id}`);

  console.log('Waiting for generation to complete...');
  const result = await pollTaskStatus(task.id);

  console.log(`Generation complete! Downloading to ${outputPath}...`);
  await downloadVideo(result.id, outputPath);

  console.log('Done!');
  return outputPath;
```

```
}

generateVideo(
  'A futuristic city at night with flying cars and neon lights, cinematic aerial shot',
  'futuristic_city.mp4',
);
```

Python

```python
import os
import time
import httpx

API_KEY = os.environ["API_KEY"]
BASE_URL = "https://your-new-api-domain.com/v1"

headers = {
    "Authorization": f"Bearer {API_KEY}",
    "Content-Type": "application/json",
}


def create_video_task(
    prompt: str,
    size: str = "1920x1080",
    duration: int = 8,
    negative_prompt: str | None = None,
    seed: int | None = None,
    generate_audio: bool | None = None,
) -> dict:
    """发起视频生成任务。路径: POST /v1/video/generations（video 单数）"""
    payload: dict = {
        "model": "veo-3.1-generate-preview",
        "prompt": prompt,
        "size": size,       # 格式 "WxH"，如 "1920x1080"
        "duration": duration,  # 顶层 duration 字段
    }

    # Veo 原生参数放在 metadata 中
    metadata: dict = {}
    if negative_prompt:
        metadata["negativePrompt"] = negative_prompt
    if seed is not None:
        metadata["seed"] = seed
    if generate_audio is not None:
        metadata["generateAudio"] = generate_audio
    if metadata:
        payload["metadata"] = metadata

    with httpx.Client() as client:
        response = client.post(
            f"{BASE_URL}/video/generations",
            headers=headers,
            json=payload,
            timeout=30,
```

```
        )
        response.raise_for_status()
        return response.json()


def poll_task_status(
    task_id: str,
    interval: float = 5.0,
    timeout: float = 300.0,
) -> dict:
    """轮询任务状态。路径: GET /v1/video/generations/:task_id（video 单数）
    状态值: queued → in_progress → completed / failed
    """
    start_time = time.time()

    with httpx.Client() as client:
        while time.time() - start_time < timeout:
            response = client.get(
                f"{BASE_URL}/video/generations/{task_id}",
                headers=headers,
                timeout=30,
            )
            response.raise_for_status()
            task = response.json()

            print(f"[{time.strftime('%H:%M:%S')}] Status: {task['status']} ({task.get('progres

            if task["status"] == "completed":
                return task

            if task["status"] == "failed":
                error_msg = (task.get("error") or {}).get("message", "Unknown error")
                raise RuntimeError(f"Task failed: {error_msg}")

            time.sleep(interval)

    raise TimeoutError(f"Task timed out after {timeout}s")


def download_video(task_id: str, output_path: str) -> None:
    """下载视频内容。路径: GET /v1/videos/:task_id/content（videos 复数）"""
    with httpx.Client() as client:
        with client.stream(
            "GET",
            f"{BASE_URL}/videos/{task_id}/content",
            headers=headers,
```

```
            timeout=120,
        ) as response:
            response.raise_for_status()
            with open(output_path, "wb") as f:
                for chunk in response.iter_bytes(chunk_size=8192):
                    f.write(chunk)


def generate_video(prompt: str, output_path: str = "output.mp4") -> str:
    """完整的视频生成流程"""
    print("Submitting video generation task...")
    task = create_video_task(
        prompt=prompt,
        size="1920x1080",
        duration=8,
        negative_prompt="blurry, low quality, distorted",
        generate_audio=False,
    )
    task_id = task["id"]
    print(f"Task created: {task_id}")

    print("Waiting for generation to complete...")
    result = poll_task_status(task_id)

    print(f"Generation complete! Downloading to {output_path}...")
    download_video(result["id"], output_path)

    print("Done!")
    return output_path


if __name__ == "__main__":
    generate_video(
        prompt="A peaceful Japanese garden with cherry blossoms falling, soft morning light, c
        output_path="japanese_garden.mp4",
    )
```

错误处理

常见错误码

HTTP 状态码	错误类型	解决方案
`400`	`invalid_request`	检查参数格式，确认 `prompt` 不为空，`size` 格式为 `WxH`
`401`	`authentication_error`	检查 API Key 是否正确、是否过期
`403`	`permission_denied`	确认账号有 Veo 3.1 访问权限
`429`	`rate_limit_exceeded`	降低请求频率，实现退避重试
`500`	`server_error`	等待后重试，或联系平台支持

退避重试示例

```
import random

def retry_with_backoff(fn, max_retries=3, base_delay=1.0):
    for attempt in range(max_retries):
        try:
            return fn()
        except httpx.HTTPStatusError as e:
            if e.response.status_code == 429 and attempt < max_retries - 1:
                delay = base_delay * (2 ** attempt) + random.uniform(0, 1)
                print(f"Rate limited, retrying in {delay:.1f}s ...")
                time.sleep(delay)
            else:
                raise
```

最佳实践

Prompt 编写技巧

1. **描述主体**：明确说明画面中的主要对象（人物、动物、场景）
2. **指定风格**：如 `cinematic`、`photorealistic`、`anime style`、`documentary`
3. **描述镜头运动**：如 `slow zoom in`、`aerial drone shot`、`tracking shot`、`handheld camera`
4. **设定光线与氛围**：如 `golden hour lighting`、`soft morning mist`、`neon-lit night`
5. **使用 negativePrompt**：排除 `blurry`、`low quality`、`watermark`、`text overlay`

推荐 Prompt 结构：

```
[主体描述], [场景/环境], [风格], [镜头类型], [光线条件], [动作/氛围]
```

性能优化

- **并发任务**：可同时提交多个任务，注意平台并发限制
- **轮询间隔**：建议初始等待 15 秒后开始轮询，之后每 5 秒一次
- **及时下载**：视频生成完成后应尽快通过 `/v1/videos/:task_id/content` 下载到持久存储
- **固定 seed**：相同 seed 的相同 prompt 结果可复现，可利用此特性做缓存

分辨率与计费

size 参数	推算分辨率	推算宽高比	4K 费率倍数（veo-3.1）
`1280x720`	`720p`	`16:9`	1.0x
`1920x1080`	`1080p`	`16:9`	1.0x
`1080x1920`	`1080p`	`9:16`	1.0x
`3840x2160`	`4k`	`16:9`	1.5x

附录：接口路径速查

操作	方法	路径
提交视频生成任务	`POST`	`/v1/video/generations`
查询任务状态	`GET`	`/v1/video/generations/:task_id`
下载视频内容（代理）	`GET`	`/v1/videos/:task_id/content`

附录：请求参数速查

```
{
  "model": "veo-3.1-generate-preview",
  "prompt": "string（必填）",
  "size": "1920x1080",
  "duration": 8,
  "image": "...（图生视频，base64格式）",
  "images": ["..."],
  "metadata": {
    "durationSeconds": 8,
    "aspectRatio": "16:9",
    "resolution": "1080p",
    "negativePrompt": "blurry, low quality",
    "seed": 12345,
    "generateAudio": false,
    "personGeneration": "allow_adult",
    "compressionQuality": "medium"
  }
}
```

文档版本：v2.0 | 适用模型：veo-3.1-generate-preview | 更新日期：2026-03-06 基于 new-api 源码 `relay/channel/task/gemini/` *验证*